When to Remember: A Preregistered Study of Failure-Memory Delivery Timing in an LLM Coding Agent

Joshua Warren
Warren Applied Labs
joshua@warrenappliedlabs.com

August 2026

Abstract

Long-term memory systems for LLM agents, including MemGPT, Mem0, and A-MEM, deliver recalled context at the start of a turn, before the agent begins working. We ask whether that delivery point, rather than memory content, limits the value of failure memories. In a preregistered, hash-bound experiment on 30 synthetic TypeScript repair tasks with deliberate trap fixes, one open-weight 35B coding model repeated its own previously observed failure in 44.2% of unassisted episodes. Injecting a matched memory of that failure at turn start failed its registered support test: repeated failures fell only 4.8 percentage points, and the task-pass estimate favored an equivalent *success* memory. Delivering the identical fact at the moment the agent proposed the matched action eliminated repetition: a confirmatory run on 18 preregistered tasks measured a 35.6-percentage-point reduction (95% CI [16.7, 55.9], relative risk reduction 100%, $p = 0.0019$), with zero invalid rows and zero protocol deviations. The effect was consistent across three measurements. Removing repeated failures did not measurably improve task completion (+1.5 pp [0.0, 4.4]), so the claim is failure-mode removal, not general improvement. Our first registration returned NOT_ESTIMABLE under its own zero-cut rule; we report both registrations in full. The task corpus, harness, registrations, and analysis artifacts are public.

1 Introduction

An agent with long-term memory will, sooner or later, face a situation it has already gotten wrong once. What its memory system does in that moment decides whether the stored experience has value. Long-term memory systems answer the question the same way: retrieve whatever seems relevant and place it in the prompt before the agent starts its turn [9, 2, 24, 27]. Retrieval quality, storage structure, and summarization policy vary widely across these systems; the delivery point does not. Systems that do intervene at tool-proposal time exist, but they deliver externally supplied safety policies rather than the agent's own episodic memory [23, 13], and no prior work we know of compares the two delivery points while holding the payload fixed.

This paper isolates the delivery point. We hold the memory itself fixed (the same failure fact, the same wording, the same rendered token count) and vary only *when* it reaches the agent: at turn start, in the conventional preamble position, or at the moment the agent proposes the specific action that failed before. The distinction matters because coding agents fail in a particular way: they take an attractive wrong path (silence the flaky-looking test, edit the shadowed config file) that a general warning delivered minutes of context earlier struggles to interrupt.

We built 30 synthetic TypeScript repair tasks around six classes of such traps, gave a frozen open-weight model a first episode in which it fell into the trap, then measured whether a memory of that failure prevented repetition in a second, scored episode. The experiment was preregistered before data collection: task splits, arms, seeds, outcome definitions, statistical procedures, support thresholds, and decision rules were fixed in a hash-bound registration that the harness verifies before any row executes.

Four results, in decreasing order of evidential strength:

1. **Action-site delivery eliminated repetition (confirmatory).** A matched failure fact delivered when the agent proposed the matched action reduced repeated failures by 35.6 pp (95% CI $[16.7, 55.9]$, relative risk reduction 100%, $p = 0.0019$) against turn-start delivery of the identical fact, on all 18 preregistered tasks with zero excluded data. The estimate and test operate on paired task-level means; descriptively, the pre-action arm repeated a known failure zero times in 270 valid episodes.

2. **Turn-start failure memory failed its registered test (confirmatory rejection).** Against a matched success memory it reduced repeated failures by only 4.8 pp $[0.7, 10.0]$ while its task-pass estimate was 6.3 pp *worse* $[-15.6, +0.7]$; the registered compound test rejected it.

3. **Prevention did not become completion (bounded null).** The confirmatory run's task-pass benefit was +1.5 pp $[0.0, 4.4]$ ($p = 0.51$). The mechanism removes a known failure mode; it does not make the agent better overall.

4. **Preregistration bound us (methodological).** Our first registration ended NOT_ESTIMABLE when a single cap-terminated episode produced an unclassifiable check and the registered rule allowed zero task cuts. The confirmatory result comes from a second registration whose scoring rule was fixed before any new data existed. We report both.

The task corpus with its trap taxonomy, the execution harness, both registration documents, the machine-readable decision rules, and the per-episode logs and statistics artifacts of every registered run are public; the analysis replays from the released episode logs byte-for-byte (Section 7). Per-attempt trace archives remain operator-local with published hashes.

2 Related work

Memory systems for LLM agents. Architectures such as MemGPT [9], Mem0 [2], and A-MEM [24] manage what an agent stores and retrieves across sessions; generative-agent simulacra [10] and cognitive-architecture framings [18] organize retrieval around recency, importance, and task structure. Benchmarks such as LoCoMo [6] measure whether retrieved memories are *correct*, and recent systems characterization work taxonomizes these designs by construction and retrieval cost [8]. Situating our design on the natural axes of memory source (the agent's own episode), trigger (fingerprint match on a proposed action), delivery point (turn start vs. action site), payload control (byte-matched fact), and outcome (repetition of a known failure), the delivery-point axis is the one this literature holds constant: retrieved context lands in the prompt before generation. That axis is our sole manipulation.

Learning from failure and experience. Reflexion stores verbal self-critiques of failed attempts for subsequent trials [17]; ExpeL distills cross-task insights from experience pools [27]; Voyager grows a skill library from successes [20]; Agent Workflow Memory induces reusable workflows [22];

Self-Refine iterates on feedback within a task [5]. These systems vary what is learned and stored. Our contribution is orthogonal and narrower: given a single, already-correct failure memory, we show its value depends on when it lands.

Position and salience effects in context. Language models use long contexts unevenly, privileging beginnings and ends [4], are measurably distracted by irrelevant context [16], and are sensitive to seemingly neutral prompt formatting [15]. These results motivate our hypothesis that a warning's distance from the decision it must influence is load-bearing, but none of them tests intervention timing in an agentic loop with a frozen, matched payload. We do not claim to isolate the mechanism; Section 6 treats candidate explanations as post hoc.

Coding agents and benchmarks. Agentic software engineering is standardly evaluated by whole-task issue resolution [3, 25, 21] over ReAct-style tool loops [26, 14], with code generation measured since HumanEval [1]. Repeating one's own recorded failure is a distinct, narrower behavior that aggregate pass rates do not surface; our tasks are built to measure exactly it.

Preregistration and statistical rigor. Preregistration is established in medicine and psychology and has been argued for in NLP [19] alongside broader reproducibility programs [11] and calls for uncertainty-aware evaluation [7]. Agent evaluations in particular accumulate researcher degrees of freedom (caps, retries, scoring judgments); our design commits every such choice to a hash-verified document before execution, and our first registration's fate (Section 3.6) shows the commitment binding.

Runtime guardrails at tool time. Sandboxed risk probing [13] and guard agents that check proposed actions against policies [23] intervene at the same point in the loop as our pre-action arm. That work targets safety policies supplied from outside; we deliver the agent's own episodic failure memory, advisory-only, and measure repetition of a specific known mistake.

3 Experimental design

3.1 Task corpus

We generated 30 self-contained TypeScript repair tasks, each a small repository with a failing behavior, an executable functional contract, and a deterministic offline checker that classifies any resulting repository state as `PASS`, a specific failure class, or indeterminate. Each task embeds one *trap*: a wrong fix that is more attractive than the correct one. Six trap classes, five tasks each:

Trap class	The attractive wrong fix
config-shadowing	edit the config file that is silently shadowed by another
flaky-looking-test	mark the failing check flaky instead of fixing the state bug
hidden-invariant	patch the symptom while violating an undocumented invariant
misleading-error-message	follow the error text to the wrong layer
stale-cache-illusion	"fix" code whose behavior is a stale cache artifact
wrong-layer-fix	patch the call site instead of the owning module

Table 1: Trap taxonomy. Within a class, no two tasks exceed 0.40 normalized Jaccard similarity on state-defining sources or check scripts.

Each task ships three run-time variants (surface renamings that preserve the trap), a trapped revision, a correct-fix revision, and a no-trap control revision. Splits were frozen before any registered run: 12 pilot tasks and 18 main tasks. There is no dev split; harness development used separately generated tasks.

A registered trap audit gates every model profile: on all 30 tasks with no memory assistance, the model must fall into the trap in at least 30% of episodes and fail to produce a correct fix in at least 50%. The profile used here audited at 40.0% trapped both times it was audited (63.3% and 70.0% non-fixed).

3.2 Two-episode protocol and outcomes

Episode 1 lets the agent fail: it attempts the task, falls into the trap, and the harness freezes that history: the failure fact text, a fingerprint of the trapping action, and repository hashes. Episode 2 is the scored episode: the agent faces the same task (per variant and seed) with an injected memory that depends on the arm.

A scored episode counts as a **repeated failure** only when both hold: (i) an episode-2 action matches the frozen trap fingerprint, and (ii) the offline checker proves the same failure class as episode 1. Text similarity alone never counts. **Task pass** requires the checker to return `PASS` within all execution caps. Steps are counted from the action trace.

3.3 Arms and matched-injection contracts

Arm	Turn-start injection	Pre-action gate
NO_MEMORY	none	off
TURN_START_FAILURE	the frozen failure fact	off
TURN_START_SUCCESS	matched twin-repo success fact	off
PRE_ACTION_FAILURE	none	advisory warning on matched action
BOTH	the frozen failure fact	advisory warning on matched action

Table 2: Arms. BOTH is registered as secondary and enters no primary test.

The **timing pair** (TURN_START_FAILURE vs. PRE_ACTION_FAILURE) delivers the same single fact with identical fact ID, citation hash, and rendered token count; only the frame differs as needed to place it at the two delivery points. The gate watches proposed actions; when a proposal matches the frozen fingerprint, it injects the fact as an advisory and lets the model choose again. On any gate error the action runs without a note, and the registered rules invalidate such an episode (this never occurred). Delivery at the action site is therefore a *package*: the matched proposal is intercepted rather than executed, the fact arrives as a direct response to it, and the model re-decides: one additional model interaction on each match. That package is inseparable from the delivery point (a warning about an action cannot arrive "at" the action after it has executed), and the registered hypothesis compares delivery packages, not a pure clock shift. The design does not include a neutral-interruption control (an interception carrying no memory), so it does not isolate the marginal contribution of the fact's content within the package (Section 5).

The **content pair** (TURN_START_FAILURE vs. TURN_START_SUCCESS) matches its two facts on path-shape and action-shape hashes, token-set Jaccard in $[0.80, 1.00]$, and a rendered token gap of at most 8 tokens and 5%.

Every arm of a cell starts from the same task commit, memory image, tools, caps, seed, and model profile, with per-arm isolated worktrees, memory directories, and sessions; start-state hashes

are recorded per row and audited for equality within cells.

3.4 Model and execution

All episodes run one frozen open-weight model profile: the Qwen-family `qwen3.5:35b` release (Q4_K_M quantization, 32,768 token context, temperature 0, no thinking mode, 4,096 max output tokens). The model is identified as an artifact by its served-model digest (`3460ffee...`) and profile hash, re-verified before every run; no technical report for this release had been published at the time of writing, so we cite the closest published family report [12] for architectural context only. Caps per episode: 12 turns, 8 tool calls, 20,480 cumulative tokens, 600 s duration, 180 s per request. Host and API faults retry up to five times after the first try; exhaustion pauses the run rather than voiding a row; a returned task result, valid or invalid, is never re-executed.

3.5 Statistical analysis

The task is the analysis unit: row outcomes are averaged over variants and seeds within each task and arm, and every test operates on the 18 paired task means. Intervals are 10,000-draw grouped percentile bootstraps (95% for primaries, 90% for the no-trap diagnostic); p-values come from 10,000-draw paired sign-flip randomization tests; the analysis seed (81) is registered. The content hypothesis uses a compound $p = \max(p_{\text{repeated-failure}}, p_{\text{task-pass}})$. Holm correction spans the registered family: two members in the first registration, one in the second. The timing support rule requires an absolute repeated-failure benefit of at least 5 pp, a point relative risk reduction of at least 30%, an interval lower bound strictly above zero, and adjusted $p < 0.05$. A pilot on the disjoint 12-task split simulated 0.8364 power for the timing test at 18 tasks (0.80 required). The pilot simulation used the first registration's two-member Holm family; the second registration's single-member family can only make the same test easier, so the transferred power estimate is conservative.

3.6 Two registrations

Registration 1 (decision rule v12) ran the full five-arm design plus no-trap controls: $18 \times 3 \times 5 \times 7 =$ 1,890 rows on seeds 1–5. It allowed zero primary task cuts: any invalid row in a compared cell cuts the whole task and voids the affected hypothesis. All 1,890 rows completed; 1,888 were valid. One of the two invalid rows (a pre-action episode that hit the cumulative token cap mid-edit, leaving a repository state the checker could not classify) sat in the timing comparison. The rule fired as written: H6-timing was NOT_ESTIMABLE despite a +38.0 pp [18.0, 59.6] exploratory estimate on the 17 complete tasks. The content hypothesis, unaffected, was decided (rejected). The registered retry rule forbids re-running any returned result, so a confirmatory timing answer required new data.

Registration 2 (decision rule v13) was fixed before any new row executed. It decides H6-timing alone on the same 18 tasks with the two timing arms and *new* seeds 6–10 (540 rows). Three changes, all registered in advance: a cap-driven unclassifiable check is scored worst-case against the pre-action arm (repeated failure in PRE_ACTION_FAILURE, no repeated failure and a pass in TURN_START_FAILURE) instead of cutting the task; the hypothesis family has one member; and the pilot's power evidence transfers by pinned artifact hashes, with the harness required to reproduce the pilot's statistics byte-for-byte before any row runs. The zero-cut rule stands for every other invalid reason. The worst-case scoring rule can only shrink the measured effect, and in the event it was never exercised: the confirmatory run had zero invalid rows.

Registration 2 executed twice, and we report both executions. The first manifest completed all 540 episodes and then crashed in the harness's post-run statistics step (an option-plumbing defect

Comparison	Status	Tasks	RF benefit	Task-pass benefit	p (adj.)	Decision
Timing (R2)	confirmatory	18/18	+35.6 pp [16.7, 55.9]	+1.5 pp [0.0, 4.4]	0.0019	**supported**
Content (R1)	confirmatory	18/18	+4.8 pp [0.7, 10.0]	−6.3 pp [−15.6, +0.7]	0.9393	rejected
Timing (R1)	exploratory	17/18	+38.0 pp [18.0, 59.6]	—	—	not estimable

Table 3: Primary results. RF benefit is the reduction in task-level repeated failure rate for the candidate arm (pre-action delivery in timing rows; turn-start failure vs. turn-start success in the content row), with 95% bootstrap intervals. The content p is the registered compound value; the R1 timing row is exploratory by the registered zero-cut rule; because its decision is not estimable, no p-value is shown for it.

caught by the analysis guard). Because runs are hash-bound to the harness that started them, that run directory was abandoned with a deviation record, and the registration was re-executed on the repaired, test-covered harness with the same registered seed set. The abandoned manifest's episode log had already been written by the crashed finalization attempts, so raw arm-level outcomes existed on disk; they were never statistically analyzed before the re-execution decision, but during supervision we observed aggregate final-state counts without arm attribution, and a skeptic need not take non-observation on faith. We therefore publish both episode logs and report both descriptively: the abandoned manifest shows 100/270 turn-start repeated failures against 0/270 pre-action (+37.0 pp episode-level); the kept, confirmatory manifest shows 96/270 against 0/270 (+35.6 pp). The kept manifest's effect is the *smaller* of the two, so selection between manifests cannot manufacture the result, and the conclusion is identical under either log. The registered analysis ran exactly once, on the final manifest.

4 Results

4.1 Primary outcomes

Table 3 contains the study. All intervals and p-values operate on the 18 paired task-level means; episode counts are descriptive. In the confirmatory run the pre-action arm repeated a known failure **zero** times in 270 valid episodes, against a 35.6% task-level repetition rate for turn-start delivery of the identical fact, a relative risk reduction of 100% (95% CI [100, 100], degenerate because the candidate numerator is zero in every bootstrap draw). Every registered support condition held with margin. The estimate is consistent across three measurements: the pilot (+37.8 pp, descriptive), the first registration's exploratory estimate (+38.0 pp on 17 complete tasks, unadjusted descriptive sign-flip statistic 0.0018), and the confirmatory run (+35.6 pp on all 18).

The content comparison is a genuine negative with structure. Turn-start failure wording did reduce repetition relative to matched success wording (+4.8 pp, interval above zero), but the registered rule demanded joint support on completion as well, and there the failure fact *cost* 6.3 pp against the success fact. Under the compound test the hypothesis was rejected. Placement, not content, is what our data supports changing.

4.2 Descriptive arm outcomes

Table 4 shows the unaggregated picture. Three observations. First, the unassisted repetition rate (44.2%) confirms the traps do what they were built to do. Second, pre-action delivery drove repetition to zero in both registrations, on disjoint seeds. Third, the highest raw pass rate belongs to turn-start *success* memory (7.04%), and BOTH (gate plus turn-start failure fact) passed less

Arm	Valid rows	Repeated failures	RF rate	Pass rate
Registration 1 (seeds 1–5)				
NO_MEMORY	269	119	44.24%	2.60%
TURN_START_SUCCESS	270	110	40.74%	7.04%
TURN_START_FAILURE	270	97	35.93%	0.74%
PRE_ACTION_FAILURE	269	0	0.00%	2.97%
BOTH	270	0	0.00%	1.48%
Registration 2 (seeds 6–10)				
TURN_START_FAILURE	270	96	35.56%	1.85%
PRE_ACTION_FAILURE	270	0	0.00%	3.33%

Table 4: Raw episode-level outcomes before task-level aggregation, descriptive only. Pass rates are low in every arm because the registered caps bind hard (Section 5).

(1.48%) than the gate alone (2.97%): whatever failure wording does to an agent's preamble, it is not obviously free even when the gate handles prevention.

4.3 Prevention is not completion

The confirmatory task-pass benefit of pre-action delivery is +1.5 pp with an interval touching zero ($[0.0, 4.4]$, $p = 0.51$). Steps were essentially unchanged (5.80 vs. 5.70 mean steps per episode). Blocking the remembered wrong path does not, in this corpus and under these caps, convert into solving the task: agents typically moved from the trap to a different unsuccessful approach rather than to the correct fix.

4.4 No-trap diagnostic

Registration 1 also measured whether the gate changes behavior when no matching trap exists (540 no-trap control rows). The steps difference was -0.026 with a 90% interval $[-0.063, +0.0037]$, well inside the registered ± 2 margin: no detectable hesitation. The pass-rate half of the diagnostic is recorded as *unresolved*: its 90% interval $[-0.74, +2.22]$ pp misses the registered ± 2 pp margin by 0.22 pp, and a registered amendment documents that this margin is finer than the design can resolve at any feasible task count. The point estimate favors the gate arm (+0.74 pp); nothing in the data suggests harm.

4.5 Cost

The two registered runs consumed 56.1M tokens of episode traffic (43.4M and 12.7M). All statistics are computed deterministically from row artifacts with zero model calls, and replay byte-identically from the immutable episode logs.

5 Threats to validity

Construct. Repeated failure is defined conservatively (fingerprint match *and* checker-proven same failure class), so our rates understate softer forms of repetition. The traps are synthetic; they compress into one file tree what real repositories spread across services. The two-episode protocol hands the gate a correct, matched memory; production recall is noisier, and our effect is conditional on the match being right.

Internal. The timing pair pins fact identity, citation hash, and rendered token count; arms are isolated per worktree, memory, and session, with audited start-state equality. The gate is advisory and fail-open, so the treatment cannot operate by blocking. The abandoned first manifest of registration 2 is handled by evidence rather than assertion: both episode logs are published, the registered analysis ran only on the final manifest, and the abandoned manifest's descriptive effect is larger than the kept one's (Section 3.6), so the only selection story available runs against the reported result. The delivery-package structure of the pre-action arm (interception plus re-decision, Section 3.3) is a construct limitation: a neutral-interruption control was not run, so the package's effect is established but the marginal effect of the memory content within it is not.

Statistical. Tasks are the unit, respecting within-task clustering across variants and seeds; tests are exact permutation analogues; the confirmatory family has one member and its p-value survives any correction. The zero-numerator relative risk reduction makes the RRR interval degenerate at 100%; the absolute benefit interval $[16.7, 55.9]$ pp is the informative bound.

External. One model, one quantization, one language, one task style, frozen caps. The binding token cap suppresses pass rates in every arm and limits what completion effects could be observed. Nothing here licenses claims about other models, real repositories, or memory systems whose retrieval quality differs from our matched setting. The corpus is public as of August 2026 and carries a contamination canary; results on models trained after that date require a contamination check before comparison.

6 Discussion

Why placement might dominate. Three post hoc candidates, none isolated by our design. Distance: long-context studies show mid-context material is used unevenly [4], and a turn-start warning is maximally distant from the decision it must influence. Competition: by proposal time the context is dominated by freshly read code and tool output, and irrelevant-context effects [16] suggest the warning loses that competition. Salience: the gate delivers the fact as a direct response to the specific proposed action, when the alternative is concrete rather than hypothetical. Separating these requires designs that vary injection distance continuously; our corpus supports such follow-ups.

Design implication for memory systems. Failure memories should be routed to a just-in-time advisory delivered against the proposed action, the position where guard agents already sit [23], rather than into the retrieval preamble. Semantic and preference context can stay at turn start; nothing here argues against preamble recall in general. The implication is a routing rule, not a new architecture: store failure memories with an action fingerprint, match at tool-proposal time, inject advisory-only, fail open. Our harness's gate is a reference implementation. The rule is established only in the tested regime: a correct, byte-matched memory of this task's own failure, matched by an exact fingerprint, on synthetic tasks. Production systems face false and missed matches, noisy retrieval, advisory fatigue under repeated triggers, and per-proposal matching cost; none of these is measured here, and each could erase the benefit.

What preregistration bought. The first registration's NOT_ESTIMABLE outcome is the strongest evidence we can offer that the protocol binds: a $+38$ pp effect with $p = 0.0036$ was set aside because one row violated a rule we wrote ourselves, and the fix was registered before new data existed rather than argued after the fact. The cost was one additional run. For agent evaluations,

where caps, retries, and scoring judgments hide many researcher degrees of freedom, we found the discipline cheap relative to what it protects, echoing arguments made for NLP broadly [19, 11].

7 Reproducibility and data availability

The task corpus (30 tasks, trap taxonomy, JSON schemas, frozen inventory hash `687615b5...`), the execution harness, both preregistration documents, the machine-readable decision rules (v12, v13), and the study report are public in the project repository at tag `h6-study-2026-08`.[1] The per-episode logs, run manifests, decision-rule copies, expected designs, and deviation logs of all four runs (both registrations, the bound pilot, and the abandoned manifest), plus the statistics, power, and audit artifacts of every run that finalized, are published as release assets on the same tag; the abandoned manifest never finalized, so its bundle carries no statistics artifact by construction.[2] The registered analysis replays from the released episode logs byte-for-byte with zero model calls via a single command. The task corpus is additionally mirrored as a dataset release with a training-contamination canary GUID.[3] Per-attempt trace archives (full agent transcripts) remain operator-local; their SHA-256 hashes are bound in the released manifests. One disclosure: the confirmatory run's replay executes under a harness newer than the one that ran the episodes (post-run report-layer fixes); the replay verifies the decision artifacts byte-for-byte and records both harness hashes side by side.

Acknowledgments

The experiment harness, execution supervision, and analysis tooling were built with substantial assistance from LLM coding agents operating under the author's direction, and every number in this paper was verified by deterministic replay against the released artifacts rather than taken from agent output. The study design, registrations, and claims are the author's, and the author takes full responsibility for all content per arXiv's authorship policy. No external funding.

References

[1] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, et al. Evaluating large language models trained on code, 2021.

[2] Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. Mem0: Building production-ready AI agents with scalable long-term memory, 2025.

[3] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues?, 2023.

[4] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173, 2024. doi: 10.1162/tacl_a_00638.

[1] https://github.com/joshuaswarren/remnic/tree/h6-study-2026-08: harness under `packages/bench/`, registrations and report under `docs/research/failure-gate/`, corpus under `packages/bench/fixtures/h6-failure-gate/`.

[2] https://github.com/joshuaswarren/remnic/releases/tag/h6-study-2026-08

[3] https://huggingface.co/datasets/joshuaswarren/h6-failure-gate-tasks

[5] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. Self-refine: Iterative refinement with self-feedback, 2023.

[6] Adyasha Maharana, Dong-Ho Lee, Sergey Tulyakov, Mohit Bansal, Francesco Barbieri, and Yuwei Fang. Evaluating very long-term conversational memory of LLM agents, 2024.

[7] Evan Miller. Adding error bars to evals: A statistical approach to language model evaluations, 2024.

[8] Yasmine Omri, Ziyu Gan, Zachary Broveak, Robin Geens, Zexue He, Alex Pentland, Marian Verhelst, Tsachy Weissman, and Thierry Tambe. Agent memory: Characterization and system implications of stateful long-horizon workloads, 2026.

[9] Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. MemGPT: Towards LLMs as operating systems, 2023.

[10] Joon Sung Park, Joseph C. O'Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. Generative agents: Interactive simulacra of human behavior, 2023.

[11] Joelle Pineau, Philippe Vincent-Lamarre, Koustuv Sinha, Vincent Larivière, Alina Beygelzimer, Florence d'Alché Buc, Emily Fox, and Hugo Larochelle. Improving reproducibility in machine learning research (a report from the NeurIPS 2019 reproducibility program). *Journal of Machine Learning Research*, 22(164):1–20, 2021.

[12] Qwen Team. Qwen2.5 technical report, 2024. Closest published technical report for the open-weight Qwen model family used here.

[13] Yangjun Ruan, Honghua Dong, Andrew Wang, Silviu Pitis, Yongchao Zhou, Jimmy Ba, Yann Dubois, Chris J. Maddison, and Tatsunori Hashimoto. Identifying the risks of LM agents with an LM-emulated sandbox, 2023.

[14] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools, 2023.

[15] Melanie Sclar, Yejin Choi, Yulia Tsvetkov, and Alane Suhr. Quantifying language models' sensitivity to spurious features in prompt design or: How I learned to start worrying about prompt formatting, 2023.

[16] Freda Shi, Xinyun Chen, Kanishka Misra, Nathan Scales, David Dohan, Ed Chi, Nathanael Schärli, and Denny Zhou. Large language models can be easily distracted by irrelevant context, 2023.

[17] Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning, 2023.

[18] Theodore R. Sumers, Shunyu Yao, Karthik Narasimhan, and Thomas L. Griffiths. Cognitive architectures for language agents, 2023.

[19] Emiel van Miltenburg, Chris van der Lee, and Emiel Krahmer. Preregistering NLP research. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 613–623. Association for Computational Linguistics, 2021. doi: 10.18653/v1/2021.naacl-main.51.

[20] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models, 2023.

[21] Xingyao Wang, Boxuan Li, Yufan Song, Frank F. Xu, Xiangru Tang, Mingchen Zhuge, et al. OpenHands: An open platform for AI software developers as generalist agents, 2024.

[22] Zora Zhiruo Wang, Jiayuan Mao, Daniel Fried, and Graham Neubig. Agent workflow memory, 2024.

[23] Zhen Xiang, Linzhi Zheng, Yanjie Li, Junyuan Hong, Qinbin Li, Han Xie, Jiawei Zhang, Zidi Xiong, Chulin Xie, Carl Yang, Dawn Song, and Bo Li. GuardAgent: Safeguard LLM agents by a guard agent via knowledge-enabled reasoning, 2024.

[24] Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang. A-MEM: Agentic memory for LLM agents, 2025.

[25] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering, 2024.

[26] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models, 2023.

[27] Andrew Zhao, Daniel Huang, Quentin Xu, Matthieu Lin, Yong-Jin Liu, and Gao Huang. ExpeL: LLM agents are experiential learners, 2023.

A Amendment log

Registration 1 carried three amendments, each registered before the data it governed: (2) the cumulative token cap rose from 16,384 to 20,480 after the pilot showed the original cap consumed by re-sent context before any task could complete, with the replacement value selected by four 30-task trap audits rather than by outcome data; (3) the content hypothesis was removed from the launch power gate after two pilots measured its power at 0.000 (it remained registered, estimated, and decided); (4) the no-trap pass-rate margin was removed from the launch gate after power analysis showed no feasible task count could resolve it (the estimate remains reported, Section 4.4). Registration 2's changes are listed in Section 3.6. Full texts ship in the repository.

B Integrity receipts

Every registered run writes a frozen manifest (expected row keys and order), per-try checkpoints, an append-only deviation log, and hash-bound statistics, power, and audit artifacts; the paper report regenerates only from these. Registration 1: 1,890 rows, deviation log empty, statistics

16fd5af1.... Registration 2 confirmatory run (h6-51d25af1ed731c35a94c28fe): 540 rows, deviation log empty, statistics 0cedc1b8..., power 2f1630c7..., audit 38e14e5a.... Pilot transfer: manifest 5bce2c0c..., power cfaec22f.... Full 64-character hashes are in the repository's study report.